

۱. کد اسمل های موجود

الف) Switch Statements

مثال:

در تابع های `processPayment` و `validatePayment`، از دستور `switch` برای تشخیص نوع پرداخت (`credit_card`، `digital_wallet` و ...) استفاده شده است.

مشکل: افزودن یک روش پرداخت جدید نیاز به تغییر در چندین نقطه از کد دارد. این موضوع باعث نقض اصل `Open/Closed` می شود.

ب) Long Method

مثال:

تابع `validatePayment` شامل بررسی های متعدد برای انواع پرداخت ها است.

مشکل: خوانایی کد کاهش یافته و `unit test` را دشوار می کند.

ج) Duplicated Code

مثال:

تابع های `processDigitalWallet`، `processCreditCard` و `processBankTransfer` ساختار مشابهی دارند (مثلاً تولید `transactionId` و چاپ پیام).

مشکل: تکرار کد، نگهداری را سخت می کند.

د) Primitive Obsession

مثال:

پارامترهایی مانند `customerInfo` و `paymentDetails` به جای استفاده از شیء های اختصاصی (مثلاً `Customer` یا `PaymentDetail`)، از `<Map<String, String>` استفاده می کنند. (واقعا نوشتن توی گوگل داک عذابه)

مشکل: احتمال خطا افزایش یافته و اعتبارسنجی پیچیده می شود.

۲. نقض اصول SOLID

الف) اصل SRP

کلاس `PaymentProcessor` سه مسئولیت متفاوت دارد:

۱. پردازش پرداخت

۲. اعتبارسنجی داده ها (`validatePayment`)

۳. لاگ تراکنش (`logTransaction`)

راه حل: تفکیک این مسئولیت ها به کلاس های مستقل مانند `PaymentValidator`، `PaymentLogger` و ...

ب) اصل OCP

مشکل: افزودن یک روش پرداخت جدید (مثلاً crypto) نیاز به تغییر در تابع های `processPayment` و `validatePayment` دارد.

راه حل: استفاده از استراتژی پترن (Strategy Pattern) برای ایجاد رابط `PaymentMethod` و کلاس های مجزا برای هر روش پرداخت.

ج) اصل DIP
مشکل: کلاس `PaymentProcessor` مستقیماً به پیاده سازی های خاص (مثلاً `processCreditCard`) وابسته است.

راه حل: وابستگی به رابط های انتزاعی (مثلاً `PaymentMethod`) به جای پیاده سازی های مستقیم.

د) اصل LSP و ISP
در کد فعلی نقض آشکاری مشاهده نشد اما با اعمال اصلاحات گفته شده این اصول بهتر رعایت خواهند شد.